

Cidades ESGInteligentes

Desafio DevOps - CI/CD, Containerização e Deploy

Integrantes: [preencher nome e RM de cada integrante]

Repositório: [link do GitHub]

1. Descrição do pipeline

Ferramenta: **GitHub Actions** (arquivo `.github/workflows/ci-cd.yml`). Imagens publicadas no GitHub Container Registry. A lógica é sequencial: cada etapa só executa se a anterior for bem-sucedida.

Etapa	Descrição
Build e Testes	mvn clean package e mvn test (4 testes automatizados: listagem, criação válida, validação e teste de integração)
Build e Push Docker	Gera imagem multi-stage e publica no GHCR com tags sha do commit e latest.
Deploy Staging	SSH no servidor, executa deploy.sh (docker compose pull/up) e smoke test em /actuator/health
Deploy Produção	Mesma imagem do staging, com aprovação manual obrigatória (Required reviewers) e smoke test

Fluxo: **push/PR** → **build** → **testes** → **imagem Docker** → **staging** → **(aprovação)** → **produção**.

Trecho do workflow

name: CI/CD - Cidades ESG

on:

```
push:
  branches: [main]
pull_request:
  branches: [main]
```

env:

```
REGISTRY: ghcr.io
IMAGE_NAME: ${ github.repository }
```

jobs:

```
build-and-test:
  name: Build e Testes
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with:
        distribution: temurin
        java-version: '17'
        cache: maven
    - name: Build
      run: mvn -B clean package -DskipTests
    - name: Testes automatizados
      run: mvn -B test
    - name: Publicar relatorio de testes
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: surefire-reports
        path: target/surefire-reports
```

docker:

```
name: Build e Push da imagem Docker
needs: build-and-test
if: github.event_name == 'push'
runs-on: ubuntu-latest
permissions:
  contents: read
  packages: write
outputs:
  image: ${ steps.meta.outputs.image }
steps:
  - uses: actions/checkout@v4
  - uses: docker/setup-buildx-action@v3
  - uses: docker/login-action@v3
    with:
```

```

        registry: ${ env.REGISTRY }}
        username: ${ github.actor }}
        password: ${ secrets.GITHUB_TOKEN }}
- name: Definir nome da imagem
  id: meta
  run: |
    IMG=$(echo "${ env.REGISTRY }}/${ env.IMAGE_NAME }}" | tr '[:upper:]' '[:lower:]')
    echo "image=IMG:${ github.sha }}" >> "$GITHUB_OUTPUT"
    echo "latest=IMG:latest" >> "$GITHUB_OUTPUT"
- uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: |
      ${ steps.meta.outputs.image }}
      ${ steps.meta.outputs.latest }}
    cache-from: type=gha
    cache-to: type=gha,mode=max

deploy-staging:
  name: Deploy Staging
  needs: docker
  runs-on: ubuntu-latest
  environment:
    name: staging
    url: ${ vars.STAGING_URL }}
  steps:
    - uses: actions/checkout@v4
    - name: Deploy via SSH
      env:
        SSH_KEY: ${ secrets.SSH_KEY }}
        SSH_HOST: ${ secrets.SSH_HOST }}
        SSH_USER: ${ secrets.SSH_USER }}
      run: |
        install -m 600 /dev/null key && echo "$SSH_KEY" > key
        ssh -i key -o StrictHostKeyChecking=accept-new "$SSH_USER@$SSH_HOST" \
          'bash -s' -- staging "${ needs.docker.outputs.image }}" < deploy/deploy.sh
    - name: Smoke test
      run: |
        sleep 20
        curl --fail --retry 5 --retry-delay 5 "${ vars.STAGING_URL }}/actuator/health"

deploy-production:
  name: Deploy Producao
  needs: [docker, deploy-staging]
  runs-on: ubuntu-latest
  # Configure "Required reviewers" no environment production (aprovacao manual)
  environment:
    name: production
    url: ${ vars.PRODUCTION_URL }}
  steps:
    - uses: actions/checkout@v4
    - name: Deploy via SSH
      env:
        SSH_KEY: ${ secrets.SSH_KEY }}
        SSH_HOST: ${ secrets.SSH_HOST }}
        SSH_USER: ${ secrets.SSH_USER }}
      run: |
        install -m 600 /dev/null key && echo "$SSH_KEY" > key
        ssh -i key -o StrictHostKeyChecking=accept-new "$SSH_USER@$SSH_HOST" \
          'bash -s' -- production "${ needs.docker.outputs.image }}" < deploy/deploy.sh
    - name: Smoke test
      run: |
        sleep 20
        curl --fail --retry 5 --retry-delay 5 "${ vars.PRODUCTION_URL }}/actuator/health"

```

2. Docker

Arquitetura: dois serviços na rede `esg-net`: **app** (Spring Boot, porta 8080) e **db** (PostgreSQL 16) com volume `pgdata` para persistência. O app aguarda o healthcheck do banco antes de subir. Configuração por variáveis de ambiente (`.env`).

Dockerfile (multi-stage)

```
# ----- Estagio 1: build -----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q clean package -DskipTests

# ----- Estagio 2: runtime (imagem enxuta) -----
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S esg && adduser -S esg -G esg
COPY --from=build /app/target/cidades-esg-*.jar app.jar
USER esg
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --start-period=40s --retries=3 \
  CMD wget -qO- http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

docker-compose.yml

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB:-esgdb}
      POSTGRES_USER: ${POSTGRES_USER:-esg}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-esg}
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - esg-net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-esg} -d ${POSTGRES_DB:-esgdb}"]
      interval: 10s
      timeout: 5s
      retries: 5

  app:
    image: ${APP_IMAGE:-cidades-esg:local}
    build: .
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    environment:
      DB_URL: jdbc:postgresql://db:5432/${POSTGRES_DB:-esgdb}
      DB_USER: ${POSTGRES_USER:-esg}
      DB_PASSWORD: ${POSTGRES_PASSWORD:-esg}
    ports:
      - "${APP_PORT:-8080}:8080"
    networks:
      - esg-net

volumes:
  pgdata:

networks:
```

esg-net:
driver: bridge

Comandos usados

```
docker build -t cidades-esg:local .  
cp .env.example .env  
docker compose up -d --build  
docker compose ps  
docker compose logs -f app  
docker compose down
```

Imagem criada: ghcr.io/<usuario>/<repositorio>:<sha> (base eclipse-temurin:17-jre-alpine, usuário não-root, HEALTHCHECK).

3. Prints do pipeline rodando

[Inserir print: pipeline - etapa de build]

[Inserir print: pipeline - testes automatizados passando]

[Inserir print: pipeline - deploy staging e produção]

4. Prints dos ambientes

[Inserir print: staging: /actuator/health e /api/cidades]

[Inserir print: produção: /actuator/health e /api/cidades]

[Inserir print: docker compose ps no servidor]

5. Desafios encontrados e soluções

Desafio	Solução
App subia antes do banco e falhava na conexão	Dependson com condition service_healthy + healthcheck pg_isready no Postg
Testes dependendo de PostgreSQL no CI	Perfil de teste com banco H2 em memória (src/test/resources).
Imagem muito grande	Build multi-stage com JRE Alpine na imagem final.
Isolar staging e produção no mesmo server	COMPOSE_PROJECT_NAME e portas distintas por ambiente (8081 / 8080).
Segredos no repositório	GitHub Secrets por environment; .env fora do git (.gitignore) e .env.example ve
Risco de subir versão errada em produção	Mãe a imagem (tag sha) do staging + aprovação manual no environment proc

Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☒
Dockerfile funcional	☒
docker-compose.yml ou arquivos Kubernetes	☒
Pipeline com etapas de build, teste e deploy	☒
README.md com instruções e prints	☐
Documentação técnica com evidências (PDF ou PPT)	☐
Deploy realizado nos ambientes staging e produção	☐